RoSo- Sumo Robots

Project Design Document

1. Design Intent

- *What experience are we trying to create?*

RoboSumo is a real-time, competitive, two-player combat game set in a physical arena. The experience merges the excitement of classic sumo wrestling with the precision and control of remotely operated robotics. Players pilot their own ESP32-powered bot from a mobile device, using Bluetooth-based joystick controls to manoeuvre, attack, and survive within the arena boundaries.

The system creates three interlocking layers of experience:

- Physical excitement: Watching two bots physically collide and push against each other in a real, tangible arena.
- Digital feedback loop: Real-time HP tracking displayed on the phone and as a NeoPixel LED health bar on each bot, creating tension and stakes.
- Competitive strategy: Players must decide when to move, when to dodge, and when to use servo arm actions, creating emergent gameplay decisions within the physical constraints of the arena.

- *Who is it for?*
  - Primary users: Two players aged 12 and above who compete against each other directly.
  - Secondary audience: Spectators watching the physical match unfold in real time.

2. Inspiration & References

- *Existing products / ideas*

Reference	What It Is	Key Insight / Learning
BattleBots (TV)	Televised robot combat competition featuring engineered bots with weapons and armour	The spectator experience is as important as the player experience. Audio and visual feedback (sparks, LEDs, crowd reaction) amplify engagement.
Sumo Wrestling	Traditional Japanese sport where competitors attempt to push each other out of a circular ring	The ring boundary as a game mechanic creates natural tension. Eliminating boundaries OR HP creates strategic duality.

- *Images / links*

https://youtu.be/d8TiGjMRuzg?si=GaobfttY_YMaTHaF

3. User Journey

Phase 1: Setup & Onboarding

1. Player 1 and Player 2 each choose their bots.
2. Each player on their phone initiates a Bluetooth pairing sequence. The app scans for and connects to the designated ESP32 of their bot (identified by name).
3. Once paired, the phone screen displays the bot's current HP and control interface.
4. The NeoPixel LED strip on each bot illuminates fully in green, indicating maximum HP (e.g., 8 bars = full health).

Phase 2: Match Begins

5. Both bots are placed inside the arena ring. A countdown or referee signal starts the match.

6. Each player uses the on-screen controls to drive their bot. The ESP32 translates control vectors into differential PWM signals, controlling left/right wheel speed for turning and forward/reverse movement.
7. Players can press a designated button on the app to trigger one or both servo arms, creating a physical pushing or blocking action.

Phase 3: Combat Interaction

8. When one bot makes physical contact with the opponent, the conductive capacitive touch film on the bot detects the change in capacitance. The ESP32 registers this as a valid 'hit' event.
9. Upon a valid hit: (a) The opposing bot's ESP32 decrements its own HP by 1. (b) The opposing bot's buzzer emits a short beep. (c) The NeoPixel LED strip on the opposing bot drops one segment, visually representing HP loss. (d) The updated HP value is transmitted back to the opponent's phone and displayed on screen.
10. The attacking bot's player sees their opponent's HP decrease on their own screen (via data feedback), reinforcing the success of the attack.

Phase 4: Match End Conditions

11. HP Depletion: When a bot's HP reaches zero, the termination sequence triggers, motors are disabled (locked state), the buzzer sounds continuously for 3 seconds, and the LED strip switches to red before going dark. The opponent is declared the winner.

Phase 5: Post-Match

13. The winning bot's LED strip pulses in a celebration pattern.
14. Players can power-cycle their bots to reset HP and begin a new match.

4. Success Criteria

Category	Criterion	Measurement / Target
Functional	Wireless control latency	Control input reflected in motor movement
Functional	Touch detection accuracy	Valid hit registered

Functional	LED response time	NeoPixel HP bar updates instantly
Functional	Buzzer feedback	Audio cue sounds for 0.5s after a hit; death buzzer sounds for the full 3 seconds
Functional	HP tracking integrity	HP decrements correctly from the initial value to zero with no skip or overflow errors
Functional	Motor disabled on death	Motors lock after death
Functional	Bluetooth range	Stable connection maintained
User Experience	Control intuitiveness	The player can navigate easily
User Experience	Spectator legibility	Observer can determine match state (HP, winner) from physical feedback alone (LED/buzzer), without seeing the phone screen
Reliability	Session stability	The full match is complete without Bluetooth disconnection or system crash
Reliability	Hit detection	No missed hits during sustained contact; no double-counting in single contact events
Reliability	Power endurance	Battery pack sustains full matches

5. System Overview

RoboSumo is a distributed embedded system consisting of two identical, symmetric bot units that operate as independent embedded platforms within a shared physical arena. Each bot is a self-contained unit with its own power supply, microcontroller, sensors, actuators, and communication hardware. No central server or PC is required during operation.

The system is best understood as three interconnected layers:

Layer	Components	Role
Control Layer	Mobile Phone + App (x2)	Player input via Bluetooth interface; displays real-time HP on screen
Processing Layer	ESP32 Microcontroller (x2)	Receives BT input, runs drive kinematics, manages game state, arbitrates hits, drives all outputs
Physical Layer	DC Motors, Servo Motors, Touch Film, LEDs, Buzzer, Battery	Locomotion, actuation, sensing, feedback, and the physical game state

Main System Components

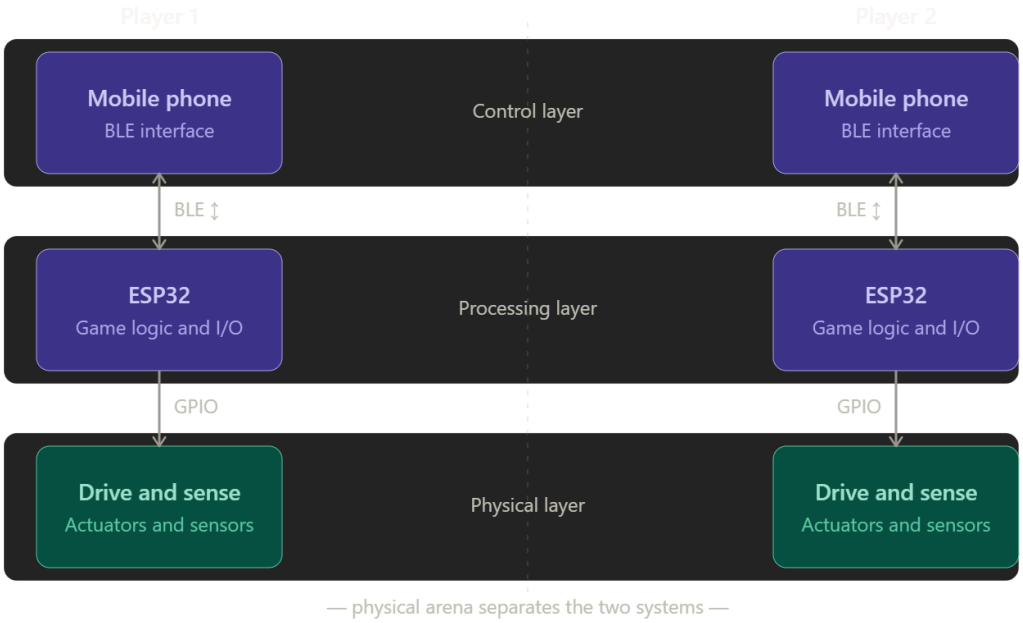
- ESP32 Microcontroller: The central processing unit of each bot. Handles Bluetooth communication, PWM generation, capacitive touch reading, game logic, and NeoPixel LED control simultaneously using its dual-core architecture.
- Bluetooth Interface: Enables real-time bidirectional communication between the mobile app and the bot , receiving button commands and transmitting HP status.

For each of them:

- DC Motors (x2): Drive the wheel system. PWM duty cycle variation controls speed and direction. A different speed between the left/right motors enables steering.
- Servo Motors (x2): Actuate the bot's mechanical arms. Controlled via precise PWM angle commands mapped from button presses (0°, 90°, 180°).
- Capacitive Touch Film: A conductive film layer on the bot's body. The ESP32 monitors changes in capacitance; a breach of the defined threshold registers a valid hit.

- NeoPixel LED Strip: A programmable RGB LED strip acting as a physical health bar. Each segment represents one HP unit; colour and animation encode game state (green = healthy, red = critical, flashing = hit received).
- Buzzer: Provides immediate audio feedback for hit events (short beep) and defeat (3-second continuous tone).
- Battery Pack: Provides independent power to each bot, enabling fully wireless, untethered operation during a match.

6. High-Level Block Diagram

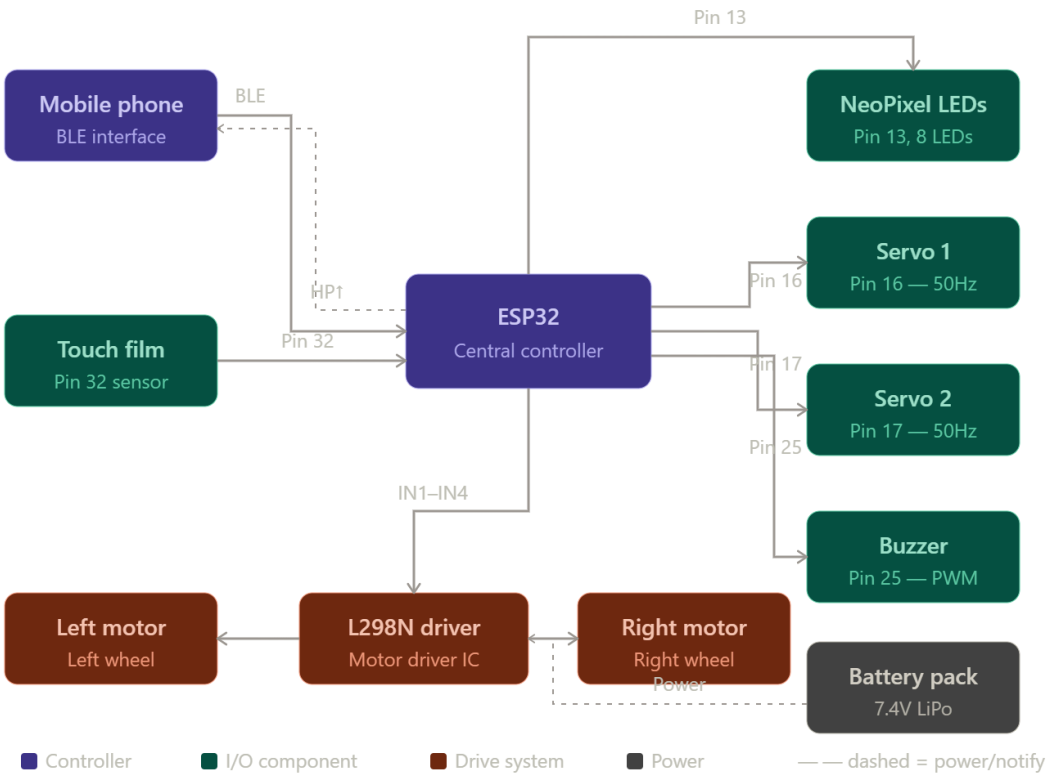


7. Input / Output Mapping

Input	Process (ESP32 Logic)	Output
Control X-axis (BT)	Map X value. Positive X = right turn (increase right PWM), Negative X = left turn.	Left & Right DC Motors

Control Y-axis (BT)	Map Y value to forward/reverse base speed. Positive Y = forward, Negative Y = reverse.	Left & Right DC Motors
Button Press AL1 (BT)	Map to Servo 1 angle command. (goes 0 to 90 degrees and back to position)	Servo Motor 1 (arm actuation)
Button Press AL2 (BT)	Map to Servo 1 angle command (goes from 0 to -90 degrees and back to position)	Servo Motor 1 (arm actuation)
Button Press AR1 (BT)	Map to Servo 2 angle command. (goes 0 to 90 degrees and back to position)	Servo Motor 2 (arm actuation)
Button Press AR2 (BT)	Map to Servo 2 angle command. (goes 0 to -90 degrees and back to position)	Servo Motor 2 (arm actuation)
Capacitive Touch Film	Read capacitance value.	HP decrement (game state update), Buzzer beep, LED HP bar update, BT → HP sent to phone
Internal HP State (HP = 0 check)	Conditional: if HP == 0, trigger termination sequence.	Motor disable (PWM = 0, locked), Buzzer 3s continuous, LED strip → red → off, BT → loss signal to phone

8. Sketches & Concepts

- *Hand sketches / early ideas*
- *Iterations*

9. Final System Architecture (Labelled)